

Lab 3 — Code Hint Sheet

Transfer Learning by Fine-Tuning — Teach a Pretrained Generator YOUR Symbol

Stuck on a **YOUR TURN** cell? Each of the three blanks is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks (and the before/after pictures) will tell you whether it is right. Only the three marked cells need editing; run everything else as-is.

Step 2 — invent your symbol. Edit the ASCII grid so '#' is ink and '.' is blank. Every row must stay the **same length**, and you want a **bold, centered** shape (roughly 10–40% ink) — thin one-pixel lines are too faint to learn.

Blank 1 — SYMBOL (the drawing).

- (a) A bold, centered shape — e.g. a heart with two round lobes on top and a point at the bottom, its widest rows a solid run of about a dozen '#'. Any heart / star / letter / initial drawn this boldly passes.
 - (b) A single thin line (one '#' per row) — too little ink (fails the $0.03 < \text{ink} < 0.6$ check, and too faint to learn).
 - (c) Every cell set to '#' (the whole grid filled) — too much ink (fails the same check; a solid block has no shape to recognize).
- (a) is the goal. (b) and (c) trip the ink-amount `assert`; rows of unequal length also error when the grid is parsed.

Step 3 — the diffusion training objective. The model was handed a noisy image and guessed the noise (`noise_pred`); the truth is `noise`. Score how far the guess is from the truth with **mean-squared error** and store it in `loss`.

Blank 2 — the loss line.

- (a) `loss = F.mse_loss(noise_pred, noise)`
 - (b) `loss = F.mse_loss(noise_pred, noisy)`
 - (c) `loss = F.cross_entropy(noise_pred, noise)`
- (a) is right — diffusion training makes the *predicted noise* match the *real noise* you added. (b) compares the guess to the noisy *image*, not the noise — wrong target. (c) cross-entropy is for class *labels*, not for matching two continuous noise images — the wrong tool. It does not error (same-shape float targets are read as probabilities), so the scalar/finite/ ≥ 0 self-checks still pass; you only see it fail *visually* at Step 6, where the **AFTER** grid never becomes your symbol.

Step 4 — the fine-tune learning rate. You are *nudging* a model that already knows how to draw, not training from scratch — so use a rate **much smaller** than a from-scratch rate. The check is $0 < \text{FINETUNE_LR} < 1\text{e-}2$.

Blank 3 — FINETUNE_LR.

- (a) `FINETUNE_LR = 1e-4`
 - (b) `FINETUNE_LR = 1e-1`
 - (c) `FINETUNE_LR = 1e-6`
- (a) is the sweet spot — big enough to pick up your symbol in ~ 100 steps, small enough not to wreck the pretrained model. (b) is far too big: it blows the model apart and the **AFTER** grid is garbage (and it exceeds the $< 1\text{e-}2$ check). (c) is so tiny the model barely moves — the **AFTER** samples still look like digits instead of your symbol.

*Answers self-check in the notebook: the symbol cell prints its ink % and shape; the objective cell prints a probe loss; and the learning-rate cell prints its value. Step 6 is a **visual** before-vs-after: watch the **AFTER** row — it should unmistakably show the symbol you drew, while the **BEFORE** row still shows the pretrained digits.*